

Homework 2

PCA for Image Compression

Suppose we have data in a matrix X which has n rows and p columns. If we let $V = X'X$ (i.e. the cross product of X), then recall that the eigendecomposition of V is

$$V = U\Lambda U'$$

where U is a $p \times p$ orthonormal matrix of eigenvectors and Λ is a $p \times p$ diagonal matrix of eigenvalues.

We can write the original data matrix X using the following identity,

$$X = XU U'$$

because U is orthonormal and $UU' = I$. If we let $Z = XU$ then we can rewrite this identity as

$$X = ZU'$$

where Z is the $n \times p$ matrix of principal component scores. If z_1 is the first column of Z (i.e. the first PC score) and u_1 is the first column of U (i.e. the first eigenvector), then we can define the *rank 1 approximation of X* as the outer product,

$$X^{(1)} = z_1 u_1'$$

which is the outer product of the vectors z_1 and u_1 . Following this pattern, the rank 2 approximation of X is $X^{(2)} = z_1 u_1' + z_2 u_2'$ by adding in the outer product of the second PC score and eigenvector. Finally, the rank p approximation of X is simply $X^{(p)} = z_1 u_1' + z_2 u_2' + \dots + z_p u_p'$ which is also equal to the original X matrix.

Each of the rank k approximations where $k < p$ can be thought of as a compressed version of the original matrix X . We can tune the amount of compression we achieve by tuning the rank of the approximation. The rank 1 approximation is the smallest version of the image we can generate using this approach and it requires $n + p$ numbers (as opposed to $n \times p$ numbers for the original matrix).

The error induced by the rank k approximation can be measured by the Frobenius norm of the difference $X - X^{(k)}$, which is

$$\text{Error}(k) = \|X - X^{(k)}\| = \sqrt{\text{trace}(X - X^{(k)})'(X - X^{(k)})}.$$

(See the `norm()` function in R for calculations.)

Image Compression Function (50%)

Download the images from Canvas and read them into R.

Write a function in R that takes as input an image (as an $n \times p$ matrix) and a value k and returns

1. The rank k approximation to the image using principal components analysis;
2. The error of the rank k approximation according to the Frobenius norm.

NOTE: In R, the `eigen()` function computes the eigendecomposition of a matrix and the `prcomp()` function does PCA directly on a matrix.

Web Site (40%)

Make a static web site that shows the code for your image compression function in a code block.

For each of the images provided,

1. A plot of the original image followed by the first 10 PCA approximations (i.e. $k = 1, \dots, 10$) for that image;
2. A plot of the approximation error vs. k for all possible values of k ;
3. A plot of the first eigenvector and the first PC score; provide an interpretation of the first eigenvector and the first PC score along with the plot;
4. Give your opinion for each image of at what value of k you think the approximation to the image is sufficiently good (you can choose a value of k larger than 10 if you want).

Publish the web site to GitHub pages and submit the URL to canvas.

Design Interactive App (5%)

Describe a App that would allow people to

1. Upload an arbitrary image;
2. Select a level of compression (i.e. a value of k);
3. View the compressed version of the image alongside the original version.

Imagine that you were going to use such an application. How would you want it to look? Where would you want various controls/buttons to be placed?

Provide an image of the mockup of your app and publish it on a separate GitHub pages web site. Submit the URL for this web site to Canvas.

Implement Interactive App (5%)

Implement the app describe in the previous section using Shiny. You can host your Shiny app on a free `shinyapps.io` account. Submit the URL to your Shiny app to Canvas.